

0:00–0:20

Practical talk: how we prepare a repository for coding agents, how we hand over a task, and how we check the result.

Outline

1	**From asking questions to delegating work**	What must we provide, and what must we verify?
2	**Environments and code checks**	Where does our code run, and what checks it?
3	**Preparing the repository**	What can the repository tell the agent?
4	**Architecture Decision Records (ADRs)**	Why does the project work this way?
5	**Version control with git**	How do we keep and review every change?
6	**Tests first**	How do tests keep an agent on track?
7	**Skills and an end-to-end workflow**	How does a request become a reviewed change?
8	**A realistic starting point**	What should I change in my repository first?

0:20–1:00

One running task carries the talk: splitting a segmentation dataset by specimen.

The eight sections answer eight practical questions.

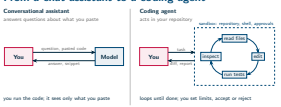
Sections 2 and 5 are the groundwork that an agent needs in any project: an environment with automatic checks, and version control.

A thread runs from the first section to the fourth: the better the repository is prepared, the shorter the request can be.

The repository and its code are illustrative; the micrographs come from our HCCI SEM dataset.

Claude Code, Codex and Matt Pocock's skills are the documented examples; the research-code examples are ours.

Tool behaviour changes quickly; the appendix lists versions and access dates.



1:10–2:20

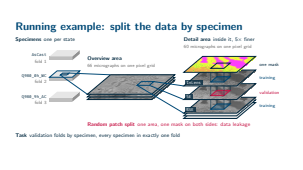
A chat assistant answers questions; we stay the integrator and carry code and results back and forth.

An agent has tools: it reads the repository, edits files, runs commands and iterates until it judges the task done.

That makes delegation possible, but it works with whatever context and permissions we give it.

Those permissions are its sandbox: the folders it may change, the commands it may run, and what it must ask us first.

Its typical failure is not a syntax error but a reasonable-looking change that violates something only we know, such as how the data must be split.



2:20–3:40

The running example uses our HCCI dataset: one specimen per material state.

On each specimen one overview area was imaged, and a detail area inside it at five times finer sampling.

Every area was imaged about 60 times: four microscopes, two or three detectors, three voltages, two dwell times.

All micrographs of an area sit on one pixel grid and share one mask, annotated once by hand.

The model trains on patches, crops of these micrographs.

A random split can put this patch of the SE micrograph in validation and the same pixels of the BSE and in-lens micrographs in training.

The model has then seen this area and its mask, so the validation score rewards recognising it: that is data leakage.

The detail area and the second etching overlap too; only the specimens share nothing.

So the task is validation folds by specimen, every specimen in exactly one fold.



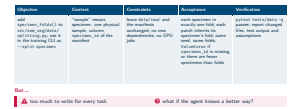
3:40–4:40

This is how we would ask a colleague who knows the project.

Three phrases each leave a part open, and two parts, constraints and verification, are missing altogether.

The agent does not know the project, so it has to answer these questions itself, and without help it guesses: where the code goes, what a sample is, what it may touch, what the test must prove and how to check the result.

A test that only checks list lengths passes even with leakage.



4:40–5:50

A bounded specification answers each question: objective, context, constraints, acceptance criteria and verification.

The manifest is the table that lists every patch with its micrograph and specimen.

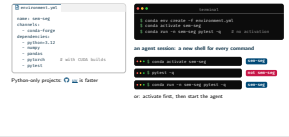
Acceptance criteria describe behaviour you can observe, including failure cases.

Click. But this is a lot to write for every task, and much of it repeats: what a specimen is, which data must not change, how to run the tests, what to report.

And by fixing every detail ourselves, we may rule out a better way that the agent knows and we do not.

The rest of the talk answers both, starting with the groundwork that every agent needs.

The classic setup: a conda environment



6:00–7:00

Most of us use conda.

An environment.yml lists channel and packages; conda builds the environment, activate enters it.

Three things matter once an agent works in it.

Each command an agent runs starts a fresh shell, so an activation does not carry over; Claude Code's documentation says to activate the environment before launching it, and conda run executes a command in an environment without activating it.

An environment.yml without versions drifts: rebuilt next year, it resolves to different packages; conda 26.5 and later export a lockfile with exact versions, conda-lock does the same for older versions.

And the channel: PyTorch's conda packages now come from conda-forge, Miniforge's only channel.

Anaconda's own channel has terms of service, free for academic use under conditions, that count AI agents and automated jobs as users; check with the university's IT before relying on it.

For Python-only projects uv is faster; A3 and A4 in the appendix show it.

Ruff, mypy and Pylance



7:00–8:00

Three tools that read the code without running it, each with its own job.

Ruff checks how the code is written: it flags likely mistakes, such as an import that is never used, and lays the code out the same way every time; a spell checker and a typesetter for code.

mypy checks whether the types match: with the hint n_folds: int, the number 3 passes and the text “3” does not, much as the units in an equation must match.

Pylance is VS Code’s language server: it completes names, shows what a function expects as you type and jumps to definitions; once its type checking is switched on, it checks types too, as you type.

Ruff and mypy run in VS Code and in the terminal, so an agent can run them too; Pylance runs only in Microsoft’s VS Code.

So we use all three: Pylance while writing, Ruff and mypy as the checks before a change counts as done.



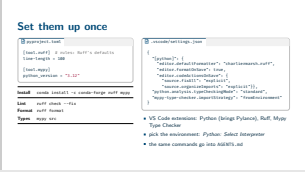
8:00–8:40

A real screenshot: VS Code 1.142 with the Ruff, Pylance and Mypy extensions on a deliberately broken version of our split function.

Squiggles mark the unused import, the undefined name and the wrong argument type; hovering shows the message and which tool raised it.

The Problems panel lists all six findings, each with its tool and rule, so you can look up or switch off a rule.

The agent sees the same findings when it runs the tools from the terminal.



8:40–9:40

Set up once per repository, then it runs by itself.

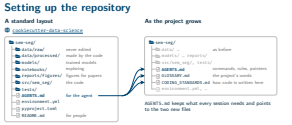
The tool settings live in pyproject.toml, which works for conda projects too.

One conda command installs both; by hand, three commands lint, format and type-check.

In VS Code, install the Python extension, which brings Pylance, plus the Ruff and Mypy Type Checker extensions, and select the project's environment.

The workspace settings make Ruff format and fix on every save, switch Pylance's type checking on, and make the Mypy extension use the mypy installed in the environment rather than the older copy it bundles.

The same commands go into AGENTS.md, so the agent runs them too.



9:50–10:50

Before any file for the agent: a layout that people and agents already know.

This one follows Cookiecutter Data Science, a template for data projects, with the code in src and its tests beside it: raw data that is never edited, processed data that the code makes, models, notebooks for exploring, figures for papers.

An agent finds its way in it the way a new colleague does.

At the root sits AGENTS.md, the one file written for the agent.

As the project grows, AGENTS.md keeps what every session needs, and two files take the rest: GLOSSARY.md for the project's words, CODING_STANDARDS.md for how code is written here.



10:50–12:10

AGENTS.md is the one file that tools supporting it load at the start of every session.

It holds what the agent cannot find out alone: the commands to set up, test and check; what the code cannot show, or which file shows it; what needs our approval; when the work is done and what to report.

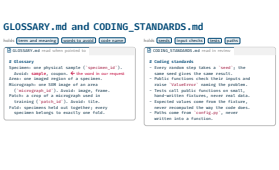
Fifteen lines, each preventing a specific mistake, and one says why: the cluster queue is shared.

The checks of section 2 are here, so the agent runs them after every edit; the boundaries are requests, not locks.

Today's models follow such files closely: OpenAI notes that its latest model is more sensitive to files such as AGENTS.md, and that unclear or conflicting guidance can make it stop early.

So leave out rules that clash or are out of date; vague wishes such as write clean code, where naming the exact mistake works better; and lines such as think carefully, which Anthropic advises removing from system prompts, since the model decides how much to think.

Keep it short, and review it like code.



12:10–13:30

Two files the agent reads only when something points to them.

A glossary holds the project's terms: what each one means, the words to avoid and the name the code uses for it; our request said sample, and here sample is a word to avoid.

The clarifying step of section 7 writes terms into it as soon as they are agreed.

Coding standards hold the rules a reviewer checks: seeds, input checks, how tests are built, where paths come from; a fixture is the small, hand-made input a test runs on.

In the workflow of section 7 the review step reads it, not the implementing agent, which already carries the most context.

Rules that a linter can check belong in Ruff's settings instead, so that every run checks them.

What an ADR is, and when to write one



13:40–14:40

An architecture decision record keeps one decision with its reasons.

Nygard's format: title, context, decision, status and consequences, including the bad ones.

Records are numbered in sequence, numbers are never reused, and a superseded record stays in the repository.

A useful test, from the skills of section 7: write one only when the decision is hard to reverse, would surprise a newcomer, and came from a real trade-off.

In research code that is usually about what a number means: the split unit, class definitions, units and normalisation, the metric.

Plot styling and renames do not need one.



14:40–15:40

The record for our running example.

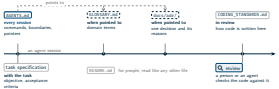
The context explains why a random patch split misleads here; the decision is short and testable; the considered options show that splitting by micrograph was rejected for the same reason.

The consequences are honest: few specimens mean few independent units, and with one specimen per material state every held-out specimen is also a new state.

ADR-0001 is not deleted; its status points to its successor.

An agent tuning for a better score finds this record before it undoes the decision.

When the agent reads each file



15:40–16:30

Each file on one agent session, at the moment it is read.

AGENTS.md at the start of every session; the glossary and the ADRs when AGENTS.md points to them.

The task specification of section 1 comes with the task; the coding standards are read when the change is reviewed, by a person or an agent.

The README is for people; the agent reads it like any other file.

Keep each fact in one place and link to it: copies go stale.



16:30–17:00

Remember the bounded specification from the start of the talk.

Every part of it was written by hand, for every task: what a specimen is, which data must not change, how to run the tests, what to report.

That is the part that repeats, and it now lives in the files of sections 3 and 4.

The same request, in a prepared repository



17:00–18:00

Now the same request, in the repository we prepared.

With these files in place, one sentence in the project's language is enough.

Four of the five parts now come from files; only the objective is in the request.

The glossary says what a specimen and a fold are.

ADR-0003 says why folds go by specimen and how they are saved.

AGENTS.md carries the boundaries, the test command and what to report.

CODING_STANDARDS.md makes seeds and input checks part of every review.

What the files leave open, here the default number of folds, is what the clarification step of section 7 asks.

This works only while AGENTS.md points to the files and they are kept current.



18:10–19:20

Many of us keep versions as copies: splitting old, v2, final, fixed.

After a few months nobody knows which copy produced a figure.

Version control replaces the copies with one folder and its history.

Git stores a snapshot of the project each time you commit, with a message, an author and a date; a diff shows what changed between two snapshots.

A branch is a separate line of work, for an experiment or for an agent's task, merged back when it is ready.

A remote such as GitHub holds a copy of the history on a server: a backup, a way to share, and the place where changes are reviewed.

There a branch comes back as a pull request: a request to merge it, with its diff open for review before it reaches main.



19:20–20:20

The learning curve is shorter than its reputation.

Five commands cover daily use: start the history once, check what changed, choose what goes into the next snapshot, commit it with a message, and push it to GitHub, the remote of the last slide.

VS Code's Source Control panel does the same with buttons: here two files are staged, the message is typed, one click on Commit, and the cloud button pushes; the graph below is the history.

The first push needs a repository on GitHub to push to; that cloud button, Publish Branch, creates one and pushes.

A .gitignore file keeps out what does not belong in the history: data, environments, which are rebuilt from their description, model checkpoints, secrets, and the worktrees that Claude Code creates.

Branches and merges can come later.



20:20–21:30

With git, an agent's work becomes a diff we can read before we accept it.

Commit before the agent starts, so there is a checkpoint to return to.

Read the diff itself, not the agent's summary of it: here, two files, every added line in green.

Undo is one command: restore discards uncommitted changes for good, revert undoes a commit and keeps the history.

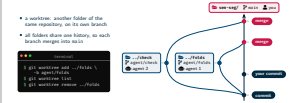
A branch, or a separate worktree, per task keeps an agent's work apart from ours until it is reviewed.

The tools assume all this.

Codex checks at launch whether the folder is under version control, suggests read-only mode when it is not, and codex exec refuses to run outside a repository unless told otherwise.

Claude Code can rewind its own file edits, but not files changed by shell commands, and its documentation says to keep using git.

The workflow of section 7 commits each finished piece of work, and its review compares against a commit.



21:30–22:30

A worktree is a second working folder of the same repository, checked out on its own branch.

Each card is one: a folder next to the repository, its branch, and the agent working in it; your own folder, sem-seg, is the main worktree on main.

Both agents start from the same commit on main and work at the same time, while you keep committing on main in your own folder.

When an agent's work is reviewed, its branch is merged into main: first one, then the other.

All worktrees share one history, and git lets a branch be checked out in only one worktree at a time, so the agents get in neither each other's way nor yours.

Three commands: add creates the folder and the branch, list shows them, remove deletes the folder once its branch is merged.



22:30–24:00

Each session can run in its own worktree, from the Claude app or from the terminal.

In the app: New or Ctrl+N, then turn on Worktree under the prompt box; it is off by default, so without it the agent edits your own checkout.

A saved environment can switch it on for every session; Settings, Claude Code, sets the worktree location and the branch prefix.

In the terminal: the flag worktree, or w, with a name; the first time, run claude once and trust the folder.

Either way each session gets its own folder under .claude/worktrees and its own branch, so the agents run side by side.

App sessions end with a pull request, a merge and Archive; uncommitted changes are kept for about thirty days.

Leaving a terminal session removes an unchanged worktree and asks about a changed one.

Tip: when a session splits into two tasks, fork it



24:00–24:50

A tip from daily use: a session often turns into more than one task.

Fork it with slash fork rather than start again: each fork keeps the whole conversation, the plan and the decisions included.

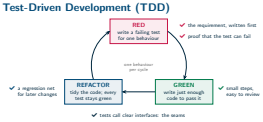
Then ask each fork to work in a worktree; Claude Code creates one under .claude/worktrees and moves the session into it.

Until then both forks work in the same folder, and their edits can collide.

In the app the forks sit under their session in the sidebar: here four tasks that came out of one session, each in its own worktree; purple marks a merged pull request.

In a terminal, slash fork copies the conversation into a new session and keeps you in this one; from a second terminal, claude with continue and fork-session copies the latest session of the folder.

Test-Driven Development (TDD)



25:00–26:00

Test-driven development in Kent Beck's formulation is a short loop.

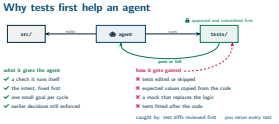
Red: write a test for one behaviour and watch it fail.

Green: write just enough code to make it pass.

Refactor: tidy the code while all tests stay green.

Then the next behaviour.

What we get: the requirement written down as a test before the code exists, proof that each test can fail, small steps, a regression net, and pressure towards clear interfaces, the seams where tests observe behaviour.



26:00–27:00

Anthropic's guidance for Claude Code puts it plainly: without a check it can run, an agent stops when the work looks done.

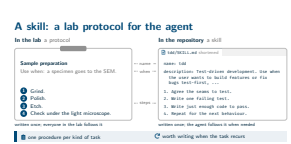
A failing test turns done into a signal the agent reads and iterates on.

Written first, from the spec, the test records intent instead of mirroring the implementation, and the suite keeps earlier decisions enforced in later sessions.

The failure modes are real: agents weaken, skip or edit tests, recompute expected values the way the code does, or replace the logic under test with a mock, a stand-in that returns whatever the test expects.

Hence our guardrails: read the changes to the tests first, and rerun every test yourself rather than trust the agent's report.

A benchmark study found that giving the model the tests with the task solved more problems; that is evidence for code generation, not for research code.



27:10–28:10

Section 3 gave the agent facts about the repository; a skill gives it a procedure.

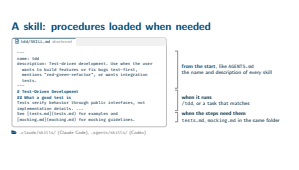
It works like a lab protocol: a name, when to use it, and the steps, written once and followed by whoever does the task.

A skill is the same for an agent: a Markdown file, here a shortened version of Pocock's test-driven development skill.

Without one, we type the same instructions into every prompt, a little differently each time.

Written once, the procedure is kept with the project, reviewed like code and improved in one place.

Worth it for a task that recurs; a one-off task needs only a good prompt.



28:10–29:20

Technically, a skill is a folder with a SKILL.md file and, where needed, more files beside it.

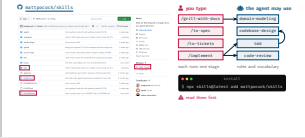
The file starts with a name and a description, which says what the skill is for and when to use it.

At the start of a session the agent sees only these lines, for every installed skill, about a hundred tokens each, just as it always sees AGENTS.md.

The rest of the file loads when the skill runs: when we type its name, here /tdd, or when the agent finds that a task matches the description.

Files beside it, here examples of good tests and rules for mocks, are read only when the steps call for them; this is called progressive disclosure.

Committed with the code, the skill reaches everyone who works in the repository.



29:20–30:10

Our example is Matt Pocock's open-source collection on GitHub; the repository uses its own AGENTS.md and GLOSSARY.md.

Its skills come in two kinds: the ones we type each run one stage of the work, and they call skills that the agent may also pick up by itself, which hold the rules.

Grill-with-docs calls domain-modeling, implement calls tdd and code-review, and tdd looks up codebase-design for its vocabulary.

The installer copies the files into the project; skills are instructions and scripts written by someone else, so read them before the agent uses them.



30:10–31:20

None of this is new: Pocock's README says the skills rest on decades of engineering experience, and quotes the books.

The Pragmatic Programmer: no one knows exactly what they want, so the agent asks before it builds; its tracer bullets return as the vertical slices of to-tickets.

Kent Beck's Extreme Programming brought the user stories that to-spec writes into every spec, and the advice to invest in the design every day.

Fowler's Refactoring catalogues code smells, which code-review looks for in every diff.

Beck's red, green, refactor loop, the subject of section 6, is the tdd skill.

Evans: one ubiquitous language for people and code, our glossary; Ousterhout: deep modules, much behaviour behind a small interface.

What is new is the reader: the model has read these books, so a term such as tracer bullets or red and green brings the whole practice into a skill in a few words.



Slide 29, continued (1 of 5): Pocock's README, "Why These Skills Exist"

I built these skills as a way to fix common failure modes I see with Claude Code, Codex, and other coding agents.

#1: The Agent Didn't Do What I Want

"No-one knows exactly what they want" — David Thomas & Andrew Hunt, The Pragmatic Programmer

The Problem. The most common failure mode in software development is misalignment. You think the dev knows what you want. Then you see what they've built - and you realize it didn't understand you at all.

This is just the same in the AI age. There is a communication gap between you and the agent. The fix for this is a **grilling session** - getting the agent to ask you detailed questions about what you're building.

The Fix is to use: `/grill-me` - for non-code uses; `/grill-with-docs` - same as `/grill-me`, but adds more goodies (see below).

These are my most popular skills. They help you align with the agent before you get started, and think deeply about the change you're making. Use them *every* time you want to make a change.



Slide 29, continued (2 of 5)

#2: The Agent Is Way Too Verbose

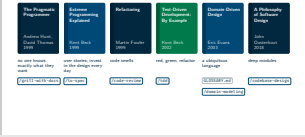
"With a ubiquitous language, conversations among developers and expressions of the code are all derived from the same domain model." — Eric Evans, Domain-Driven-Design

The Problem: At the start of a project, devs and the people they're building the software for (the domain experts) are usually speaking different languages.

I felt the same tension with my agents. Agents are usually dropped into a project and asked to figure out the jargon as they go. So they use 20 words where 1 will do.

The Fix for this is a shared language. It's a document that helps agents decode the jargon used in the project.

Example: here's an example glossary, from my `course-video-manager` repo. Which one is easier to read? **BEFORE**: "There's a problem when a lesson inside a section of a course is made 'real' (i.e. given a spot in the file system)". **AFTER**: "There's a problem with the materialization cascade". This concision pays off session after session.



Slide 29, continued (3 of 5)

#2, continued

This is built into `/grill-with-docs`. It's a grilling session, but that helps you build a shared language with the AI, and document hard-to-explain decisions in ADR's.

It's hard to explain how powerful this is. It might be the single coolest technique in this repo. Try it, and see.

Tip: a shared language has many other benefits than reducing verbosity: **variables, functions and files are named consistently**, using the shared language; as a result, the **codebase is easier to navigate** for the agent; the agent also **spends fewer tokens on thinking**, because it has access to a more concise language.



Slide 29, continued (4 of 5)

#3: The Code Doesn't Work

"Always take small, deliberate steps. The rate of feedback is your speed limit. Never take on a task that's too big." — David Thomas & Andrew Hunt, The Pragmatic Programmer

The Problem: Let's say that you and the agent are aligned on what to build. What happens when the agent *still* produces crap?

It's time to look at your feedback loops. Without feedback on how the code it produces actually runs, the agent will be flying blind.

The Fix: You need the usual tranche of feedback loops: static types, browser access, and automated tests.

For automated tests, a red-green-refactor loop is critical. This is where the agent writes a failing test first, then fixes the test. This helps give the agent a consistent level of feedback that results in far better code.

I've built a **`/tdd` skill** you can slot into any project. It encourages red-green-refactor and gives the agent plenty of guidance on what makes good and bad tests.

For debugging, I've also built a **`/diagnosing-bugs`** skill that wraps best debugging practices into a disciplined loop, gated phase by phase.



Slide 29, continued (5 of 5)

#4: We Built A Ball Of Mud

"Invest in the design of the system every day." — Kent Beck, Extreme Programming Explained

"The best modules are deep. They allow a lot of functionality to be accessed through a simple interface." — John Ousterhout, A Philosophy Of Software Design

The Problem: Most apps built with agents are complex and hard to change. Because agents can radically speed up coding, they also accelerate software entropy. Codebases get more complex at an unprecedented rate.

The Fix for this is a radical new approach to AI-powered development: caring about the design of the code.

This is built in to every layer of these skills: `/to-spec` quizzes you about which modules you're touching before creating a spec.

And crucially, `/improve-codebase-architecture` surveys a codebase for deepening opportunities and hands you the candidates. I recommend running it on your codebase once every few days. It is a survey, not a rescue: on a genuinely old codebase it will find real candidates, but it won't untangle the mud for you.

Summary: Software engineering fundamentals matter more than ever. These skills are my best effort at condensing these fundamentals into repeatable practices, to help you ship the best apps of your career. Enjoy.



31:20–32:30

The main flow as the author documents it: clarify with grill-with-docs, write the spec, cut tickets, implement each ticket.

In today's terms this is spec-driven development: an approved spec before any code, then each ticket built test-first.

Tickets are vertical slices: each a thin path through the whole program, from the command line to the output, usable on its own.

Implement drives the tdd skill at the test seams agreed in the spec, the public functions and command-line options that the tests call; then it runs code-review and commits.

Code-review checks the repository's coding standards and the spec, and is where refactoring is proposed.

Red marks the decisions that stay with us at every stage.

The author's context advice: one session from clarification to tickets, then a fresh session per ticket that starts from the written ticket.



32:30–33:50

What grilling looks like on our request; the session is abridged and written for our example in the format of Pocock's skills.

Each round asks every question that can be asked now, numbered, each with a recommended answer (2), worded so that yes accepts it.

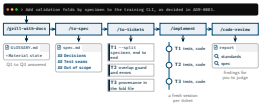
The agent looks up facts itself, here the number of specimens in the manifest (1), and leaves the decisions to us.

When a term settles, it goes into the glossary at once (the arrow); here the session finds that every specimen is also the only one of its material state, which matters for what validation can claim.

It offers an ADR only when a decision is hard to reverse, surprising and a real trade-off, so the number of folds gets none (3).

Pocock's video of July 2026 shows a full session on his own project.

The specimen split, stage by stage



33:50–35:00

The whole flow on our task, starting from the short request.

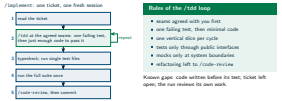
Each stage leaves something we can open and check.

Grilling settles what the repository leaves open: the default number of folds, and what validation is meant to show with only three specimens.

The spec records the decisions and the test seams.

Tickets are thin vertical slices, each usable on its own: first the whole path from the command line to a fold file, then the overlap guard and the error cases, then the provenance stored in the fold file.

Each ticket is implemented test-first in a fresh session, and review checks the standards and the spec.



35:00–36:10

In Pocock's skills, implement is the build step and you invoke it yourself, one ticket per fresh session.

It reads the ticket, drives the tdd skill at the agreed seams one red-green slice at a time, typechecks and runs single test files as it goes, runs the full suite once at the end, calls code-review and commits.

The tdd skill is a set of rules: seams agreed first, red before green, one vertical slice per cycle, tests through public interfaces, mocks only at system boundaries.

The loop has no refactor step, because agents almost never do it; code-review proposes refactoring instead.

The author's documentation lists the gaps: the agent sometimes writes code first, implement does not close the ticket, and the review inside the same run reviews its own work, so a fresh session makes a better reviewer.



36:10–36:50

Not every change needs five stages.

Two questions decide: how large or open is the change, and does it change what a result means?

Small changes that leave results unchanged go straight to the agent; we still read the diff.

A small change of meaning, such as pixel-size units, deserves a short spec.

A large routine refactor benefits from tickets.

The full flow earns its overhead when both hold, as for a new split unit, loss or metric.

Pocock's own routing asks whether the build spans several sessions; the axis of meaning is ours.



37:00–38:00

Pocock's collection is one of several; two others are worth a look.

Superpowers, by Jesse Vincent, is a whole method, from brainstorming to finishing the branch; its test-driven skill allows no production code without a failing test.

The Karpathy skills are one CLAUDE.md with four rules, written by Jiayuan Zhang from Andrej Karpathy's notes on how models go wrong in code; the last rule turns a task into a goal that can be checked, such as a test that reproduces a bug.

They differ in how much of the process they take over; Pocock's README prefers small skills you combine to systems that own the process.

What they share is section 6: the check is decided before the code.

As with any skill, read them before installing.



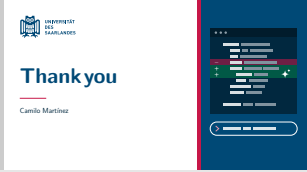
38:00–38:50

Groundwork: a git repository with a commit before every agent task, the Python environment with conda or uv, and Ruff and mypy run after every edit.

Context: AGENTS.md, the glossary, the coding standards and ADRs, so that requests can stay short.

Workflow: choose one that suits you, Pocock's skills for example; write your own skills for the tasks you repeat; agree the tests before the code; give each task its own worktree or forked session; and review the code and the science yourself.

None of this needs a new tool, and most of it helps human collaborators too.



38:50–39:20

Thank you.

Questions to open the discussion: which of your projects is not under git yet; which repository you would try this on first, and who reviews it; which decision in your project deserves an ADR; which single command tells you that your code still works.

Templates, tool details and references follow in the appendix.

A3 The upgrade: uv



If asked

uv, from Astral, the makers of Ruff, is one tool for what pip, venv, pip-tools, pipx and pyenv do separately.

A project is a pyproject.toml with what the project needs and a uv.lock with the exact version of every package for every platform; commit both.

uv builds the environment in .venv inside the project; it is not committed and can be rebuilt at any time.

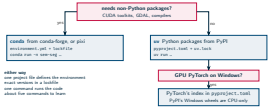
uv add adds a package and updates the lockfile; tools for development, such as pytest, Ruff and mypy, go into their own group.

uv run checks that lockfile and environment are in sync and then runs the command, so there is nothing to activate, which suits an agent whose commands each start a fresh shell.

Astral reports 10 to 100 times the speed of pip.

The current version is 0.13; uv has not reached 1.0, but Astral treats it as stable.

A4 conda or uv?



If asked

One question decides: does the project need packages that are not Python?

conda installs packages of any language from conda-forge, including CUDA toolkits, GDAL or compilers; uv installs Python packages from PyPI and nothing else.

Both can now pin exact versions: uv with its lockfile, conda with its own lockfile since version 26.5 or with conda-lock.

conda keeps environments in a central folder, uv in the project.

For GPU work, conda-forge has CUDA builds, while PyPI's PyTorch wheels for Windows are CPU-only, so a uv project needs PyTorch's own index in its pyproject.toml, as the uv guide shows.

The learning curve is small: a handful of commands, and the habit that the project file, not the shell, defines the environment.

If a project needs non-Python packages, stay with conda, or look at pixi, which installs conda-forge packages with a lockfile and uses uv for PyPI packages.